

RBAC technology stack

Companion to the project brief

The project brief covers what we are building and why. This document covers what we are building it with, and the two places where this particular stack needs care.

Stack at a glance

Layer	Choice
Admin console	Next.js (TypeScript)
API	NestJS (TypeScript)
Database	Microsoft SQL Server on Amazon RDS
Data access	TypeORM (<code>mssql</code> / <code>tedious</code> driver)
Compute	AWS Lambda (Node.js 22)
API edge	API Gateway (HTTP API)
Console hosting	S3 + CloudFront
Secrets	AWS Secrets Manager
Infrastructure as code	AWS CDK
Client SDK	private npm package (AWS CodeArtifact)

Authentication is not ours — the existing identity provider issues the token, and we verify it.

Repository layout

One monorepo. The SDK living next to the API is the point: the contract cannot drift.

```
apps/  
  console/      Next.js admin UI  
  api/          NestJS service  
packages/  
  contracts/    shared DTOs and permission-string helpers  
  sdk/          client library the other applications install  
infra/          AWS CDK stacks
```

Frontend - Next.js

This is an internal admin console. No SEO, no public traffic, no server-rendered content that matters. So **build it as a static export** (`output: 'export'`) and serve it from S3 behind CloudFront. That removes SSR, Lambda@Edge and a whole category of deployment problems we would otherwise be debugging for free. If we ever need SSR we can move to Amplify Hosting later.

Suggested libraries: TanStack Query for server state, react-hook-form with zod for forms, and the zod schemas imported from `packages/contracts` so validation matches the API exactly.

Screens to build: Applications, Screens & Actions (read-mostly, populated by manifest sync), Roles, Users, Assignments, Audit log.

The one genuinely hard piece of UI is the **role builder** — a screens x actions matrix with parent/child screen rows, tri-state checkboxes on parents, and bulk select per column. Budget real time for it; everything else is standard CRUD.

Backend - NestJS

Nest modules map directly onto the domain: `applications`, `resources`, `actions`, `permissions`, `roles`, `assignments`, `users`, `audit`, plus `manifest` and `runtime`.

There are **two API surfaces with different authentication**, and they should not be blurred:

Surface	Caller	Auth
Admin API	a human in the console	user JWT from the IdP
Machine API	an application's CI pipeline or backend	client credentials per application

Manifest sync is on the Machine API. `GET /v1/me/permissions` is on the Admin-style surface but called by the other applications' backends on behalf of a user.

Lambda adapter. Use `@codegenie/serverless-express` and cache the bootstrapped Nest app *outside* the handler function so warm invocations reuse it:

```
let cached: Handler;

export const handler = async (event, context) => {
  cached ??= await bootstrap(); // created once per execution environment
  return cached(event, context);
};
```

Keep `main.ts` (local HTTP server) and `lambda.ts` (adapter) as two thin entrypoints over the same `AppModule`. Bundle with **esbuild**, not `webpack` — bundle size is cold-start time.

Database - SQL Server

RDS for SQL Server in private subnets, Multi-AZ in production. Credentials in Secrets Manager, never in environment variables.

- **TypeORM over Prisma here.** Prisma's SQL Server connector is fine, but it ships a query-engine binary that inflates the Lambda bundle significantly. TypeORM on tedious is lighter and has first-class Nest integration.
- **Migrations are checked into the repo and run as a separate, gated CI step** — never on application boot, and never on Lambda cold start. Concurrent Lambdas racing to migrate is a bad afternoon.
- `synchronize: true` must be off everywhere, including dev, once the first migration exists.
- Add a `permission_version` counter, bumped in the same transaction as any role or assignment change. That integer is what the SDK caches against (see the brief).

AWS deployment

```
CloudFront —> S3 (console, static)
CloudFront —> API Gateway —> Lambda —> RDS SQL Server (private subnets)
                                         |
                                         —> Secrets Manager, CloudWatch
```

Lambda has to sit **inside the VPC** to reach RDS. That means VPC endpoints for Secrets Manager and CloudWatch Logs, or a NAT gateway — the endpoints are usually the cheaper answer. Flag this early because it affects the CDK stack, not just a config value.

Split into two Lambdas, not one:

Lambda	Traffic	Sizing
<code>runtime</code> — the permission-fetch endpoint	hot, latency-critical	small bundle, provisioned concurrency
<code>admin</code> — everything the console calls	low volume, cold-start tolerant	plain on-demand

Environments and CI/CD

- Three environments: `dev`, `staging`, `prod` — separate AWS accounts if we can, separate stacks at minimum.
- GitHub Actions: `lint`, `typecheck`, `test`, `build`, `cdk deploy`. Migrations are a manual-approval step on staging and prod.
- The SDK is versioned and published to CodeArtifact on tag. Consuming applications pin a version; they do not track `latest`.